

Circuits

Tracing an algorithm inside GPT-2 · video 2:16

This lesson answers one question: can we find *which parts* of a model carry out a task, and in what order? The test case is **indirect object identification** (IOI): “When Kate and Peter went to the store, Peter gave a drink to” → Kate. With **activation patching** we watch the information move from the repeated name to the last position at layers 7–8, find a handful of attention heads that write the answer, and see that knocking them out hurts but does not break the behaviour: real circuits have backups.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 8).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The task and the metric	▶ 0:08-0:34	2	☐
2. Activation patching through the residual stream	▶ 0:34-1:07	3	☐
3. Heads and the circuit	▶ 1:07-1:37	2	☐
4. Knockout and redundancy	▶ 1:37-2:01	3	☐

Before you start: Deep Dive episodes 13 (the residual stream), 8 (the attention matrix) and 42 (induction heads). Code: `code/d45_circuits` (PyTorch + transformers, a few minutes on a CPU).

CONCEPT 1 OF 4

The task and the metric

▶ Video 0:08–0:34

In “When Kate and Peter went to the store, Peter gave a drink to”, the right next word is the name that was *not* repeated: Kate, the indirect object. We test GPT-2 small on 90 such prompts (3 sentence templates × 30 name pairs). It prefers the right name on all 90.

which parts do what, and in what order

indirect object identification (IOI)

▶ 0:17 The task: indirect object identification (IOI).

When Kate and Peter went to the store, Peter gave a drink to

→ Kate
the name that wasn't repeated

90 prompts: right name 100% · $\text{logit}(\text{Kate}) - \text{logit}(\text{Peter}) = 2.68$

▶ 0:32 90 prompts: GPT-2 prefers the right name 100% of the time; mean logit difference 2.68.

Instead of accuracy we track a continuous score, the **logit difference**: $\text{logit}(\text{right name}) - \text{logit}(\text{repeated name})$, at the last position. Its mean is 2.68: the right name is $e^{2.68} \approx 15$ times likelier than the wrong one. A continuous metric shows partial effects that accuracy would hide.

KEY IDEA

Pick a narrow task the model does reliably, and a continuous metric for it: here, $\text{logit}(\text{IO}) - \text{logit}(\text{S})$.

Check 1: The task and the metric

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

A logit difference of 2.68 between two names: how many times likelier is the right name than the repeated one?

.....

1.2 SHORT ANSWER

Why use the logit difference instead of “right name or not”?

.....

CONCEPT 2 OF 4

Activation patching through the residual stream

▶ Video 0:34–1:07

Make a **corrupted** prompt: "... Kate gave a drink to". Now the right answer is Peter, and the mean logit difference (Kate – Peter) is -3.30 . Run the corrupted prompt but overwrite one activation with its value from the clean run, and measure how much of the clean answer comes back:

$$\text{recovered} = (\text{patched} - \text{corrupted}) / (\text{clean} - \text{corrupted})$$

1 means that activation alone carries everything that distinguishes the two runs; 0 means it carries nothing.

When Kate and Peter went to the store, Peter gave a drink to
clean: answer Kate (+2.68)

When Kate and Peter went to the store, Kate gave a drink to
corrupted, second name swapped: answer flips (-3.30)

copy one activation

$\text{recovered} = (\text{patched} - \text{corrupted}) / (\text{clean} - \text{corrupted})$

▶ 0:49 Corrupted prompt: the second name swapped, so the answer flips (-3.30); copy in one activation from the clean run.

patch the residual stream: share recovered

	Kate	Peter	Peter (2nd)	to (last)
0	0.00	0.00	1.00	-0.00
1	0.00	0.00	1.00	0.00
2	0.00	0.00	1.00	-0.00
3	0.00	0.00	0.99	0.00
4	0.00	0.00	0.96	-0.00
5	0.00	0.00	0.96	0.00
6	0.00	0.00	0.91	0.03
7	0.00	0.00	0.65	0.41
8	0.00	0.00	0.03	0.98
9	0.00	0.00	-0.02	1.00
10	0.00	0.00	0.00	0.97
11	0.00	0.00	0.00	1.00

through layer 6:
at the repeated name

layers 7–8:
moves to the last position

▶ 1:06 Patching the residual stream: through layer 6 what matters sits at the repeated name; at layers 7–8 it moves to the last position.

LAYER	KATE	PETER	PETER (2ND)	"TO" (LAST)
0–3	0.00	0.00	1.00–0.99	0.00
6	0.00	0.00	0.91	0.03
7	0.00	0.00	0.65	0.41
8	0.00	0.00	0.03	0.98
9–11	0.00	0.00	≈ 0	0.97–1.00

(Template 1, 30 prompts.) The first two names are identical in both runs, so patching them does nothing. The difference starts at the second name; layers 7 and 8 read it and move the result to the last position, where the answer is written. Patching shows *where* and *at what depth* the information lives.

KEY IDEA

Activation patching asks: if I restore just this piece, does the behaviour come back? It locates information in position and depth.

Check 2: Activation patching through the residual stream

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Clean 2.68, corrupted -3.30 . A patch gives a logit difference of 0. What share is recovered?

.....

2.2 TRUE OR FALSE

"Patching the first 'Kate' recovers 0.00, so GPT-2 ignores the first name."

True ☐ False ☐

2.3 SHORT ANSWER

Why does the recovered share at the repeated name fall from 0.91 at layer 6 to 0.03 at layer 8, while at the last position it rises to 0.98?

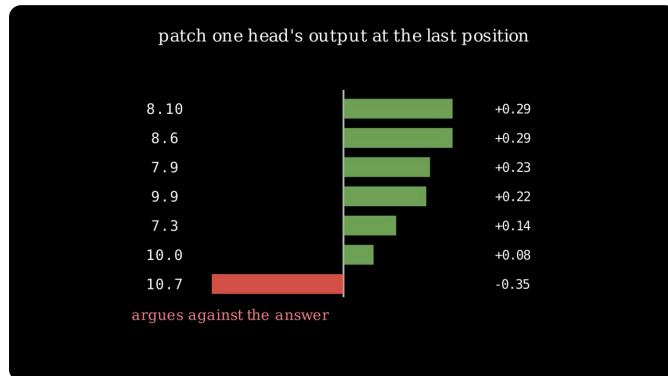
.....

CONCEPT 3 OF 4

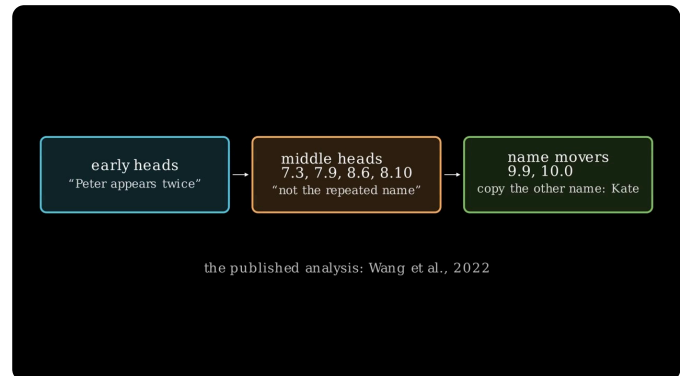
Heads and the circuit

▶ Video 1:07-1:37

Patch each of the 144 heads (layer.head) at the last position. A handful matter: 8.10 and 8.6 recover 0.29 each, 7.9 0.23, 9.9 0.22, 7.3 0.14, 10.0 0.08. One head, 10.7, gives -0.35 : restoring it makes the answer *worse*; it argues against the answer the rest of the circuit writes.



▶ 1:22 Patching single heads at the last position: 8.10, 8.6, 7.9, 9.9 recover about a quarter each; 10.7 pushes the other way.



▶ 1:36 The story from Wang et al., 2022: early heads, middle heads, name movers.

The published analysis (Wang et al., 2022) explains the roles. **Early heads** notice that a name appears twice. **Middle heads** (S-inhibition: 7.3, 7.9, 8.6, 8.10) pass on “not the repeated name” to the last position. **Name movers** (9.9, 10.0 and others) attend to the remaining name and copy it to the output. Our patching finds the same middle and late heads; the full role analysis is in the paper.

KEY IDEA

A circuit is a small set of components, each with a role, composed in order: detect the duplicate, inhibit it, copy the other name.

Check 3: Heads and the circuit

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 PUT IN ORDER

Put the circuit's steps in order.

3.2 SHORT ANSWER

What does a negative recovered share (10.7: -0.35) mean?

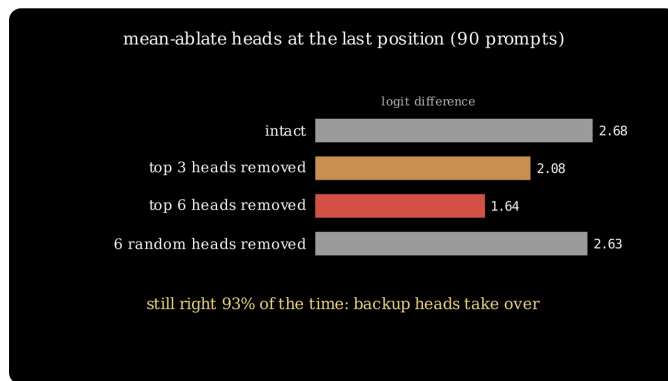
CONCEPT 4 OF 4

Knockout and redundancy

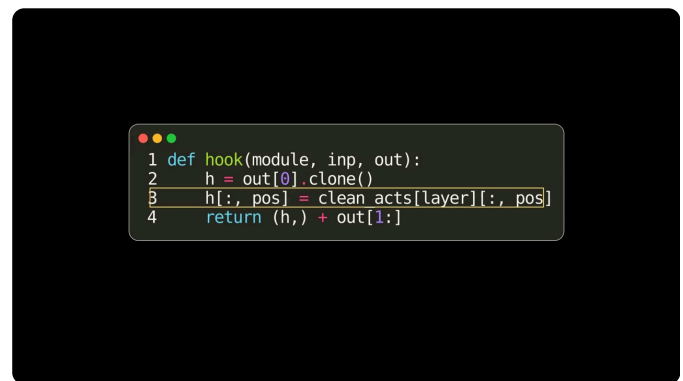
▶ Video 1:37-2:01

Patching shows what is *sufficient* to restore the answer; **knockout** tests what is *necessary*. Replace a head's output at the last position by its average over all 90 prompts (mean ablation), so it carries no prompt-specific information.

HEADS REMOVED	LOGIT DIFFERENCE	RIGHT NAME
none	2.68	100%
top 3 (8.10, 8.6, 7.9)	2.08	100%
top 6 (+ 9.9, 7.3, 10.0)	1.64	93%
6 random heads (mean of 5 draws)	2.63	



▶ 1:52 Mean-ablating heads at the last position: top 6 → 1.64, still right 93%; 6 random heads → 2.63.



▶ 2:00 Patching in code: a forward hook that overwrites one activation.

The chosen heads matter far more than random ones, yet GPT-2 still answers right 93% of the time: **backup heads** take over when the main ones are gone. Circuits found by patching are real but rarely the whole story; the behaviour is spread over redundant paths.

KEY IDEA

Patching finds what is sufficient; knockout tests what is necessary. Real circuits are redundant.

Check 4: Knockout and redundancy

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

What fraction of the logit difference is lost when the top 6 heads are removed?

4.2 MULTIPLE CHOICE

Why compare with removing 6 random heads?

- A. To check the effect is about these heads, not about removing any 6 heads
- B. Random heads are the backup heads
- C. To make the numbers smaller

4.3 HANDS-ON CODE

Try it yourself. Patch one block's output at one position with a forward hook.

```
def patch(layer, pos, clean_acts):
    def hook(module, inp, out):
        h = out[0].clone()
        h[:, pos] = clean_acts[layer][:, pos]
        return (h,) + out[1:]
    return model.transformer.h[layer].register_forward_hook(hook)

handle = patch(8, -1, clean_acts)      # clean_acts saved by a hook during the clean
run
logits = model(corrupted_ids).logits
handle.remove()
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The task and the metric · rewatch ▶ 0:08-0:34

1.1 About 14.6. The ratio of two softmax probabilities is $e^{(\text{difference})}$: $e^{2.68} \approx 14.6$.

1.2 It is continuous, so it shows partial damage or partial recovery; accuracy only changes when the top answer flips. In part 4, removing six heads cuts the difference by 39% but accuracy only from 100% to 93%.

2. Activation patching through the residual stream · rewatch ▶ 0:34-1:07

2.1 About 0.55. $(0 - (-3.30)) / (2.68 - (-3.30)) = 3.30 / 5.98 \approx 0.55$.

2.2 False. The first name is the same in the clean and corrupted prompts; patching only measures what differs between the two runs.

2.3 Attention heads at layers 7-8 copy the relevant information from the repeated-name position to the last position; after that, the last position already holds it. Information moves between positions only through attention.

3. Heads and the circuit · rewatch ▶ 1:07-1:37

3.1 early heads detect the repeated name → middle heads pass on “not the repeated name” → name movers copy the remaining name. Each step needs the previous one's output, which is why they sit at increasing depth.

3.2 Restoring that head's clean output moves the result further from the clean answer: the head pushes against the answer the circuit is writing. Wang et al. call such heads negative name movers; they reduce overconfidence.

4. Knockout and redundancy · rewatch ▶ 1:37-2:01

4.1 About 39%. $(2.68 - 1.64) / 2.68 \approx 0.39$.

4.2 To check the effect is about these heads, not about removing any 6 heads. Random heads drop the difference only from 2.68 to 2.63.

4.3 With layer 8 and the last position, about 98% of the clean logit difference comes back (template 1). This is part 2 of the episode's code: by layer 8 the answer sits at the last position.